

Make assert() macro user friendly for C and C++

Document #: P2264R7 (WG21)/ (was N2829, N2621) (WG14)
Date: 2023-11-10
Project: Programming Language C++
Audience: WG21 - Library (WG14 adopted for C23)
Reply-to: Peter Sommerlad
<peter.cpp@sommerlad.ch>

Contents

- 1 History
 - 1.1 R7 revision
 - 1.2 R6 revision
 - 1.3 R5 revision
 - 1.4 R4 revision
 - 1.5 R3 revision
 - 1.6 R2 revision
 - 1.7 R1 revision
 - 1.8 R0 initial revision
- 2 Introduction
- 3 Remedy
- 4 Impact on existing code
- 5 Potential liabilities of the proposed change
 - 5.1 NDEBUG will allow `assert()` without any arguments
 - 5.2 “Contracts will make the 50 year old `assert` macro obsolete and to not suffer from the macro parsing issue.”
 - 5.3 “Using the comma operator can be misapplied to an always true assert, if its arguments are formed as `static_assert(condition, reason)`. This will make wrong code compile that today doesn’t.”
 - 5.4 “Teachability is not improved, because we can teach use extra parentheses today.”
 - 5.5 Will changing `static_assert(condition, "reason")` to `assert(condition, "reason")` compile and silently make the assert never fire. (see 2 above)
 - 5.6 Do you have implementation experience?
 - 5.7 Why don’t you use `bool(__VA_ARGS__)` to prevent comma operator usage?
 - 5.8 Why don’t you use `assert(arg, __VA_ARGS__)` to prevent zero arguments for the NDEBUG case?
 - 5.9 With the suggested static cast to bool, an enum class type would be viable as truth values, this contrasts their behavior in other boolean contexts
 - 5.10 To be discussed by LWG:
- 6 Wording for C++ (to be voted on)
 - 6.0.1 19.3.2 The `assert` macro [`assertions.assert`]
- 7 obsolete ~~Wording for C++ if C is changed~~

- 8 Wording for C
 - 8.0.1 7.2.1 Program diagnostics
- 9 Acknowledgements
- 10 Example implementation (BSD license on MacOS)
 - 10.1 C test cases
 - 10.2 C++ test cases

§ 1 History

§ 1.1 R7 revision

Change of wording according to wording review by LWG 20231110.

§ 1.2 R6 revision

Thanks to splendid input by Daveed Vandervoorde and Jonathan Wakely, suppressing the conversion of scoped enums no longer requires any further dependency, e.g., `<type_traits>`, so that there is no longer any LEWG design issue to be taken. This has the benefit that `assert()` does no longer compile in C++ as well, except when `NDEBUG` is set.

One thing that remains open: Do we want to wait for an omnibus C23 adaptation paper to incorporate the changes to `assert(...)` (decision in LWG 20231110 no), use wording of section 6 of this paper.

§ 1.3 R5 revision

Added some LEWG discussion and my rebuttal to it. Make static cast to bool explicit also in the wording. AFAIK, this only makes a difference from contextual conversion to bool for enum class types.

There remains a single question with LEWG to definitely answer: Are you OK with static cast to bool, allowing enum class values as arguments to `assert()` to compile, OR, do you want prevent that with ugly compile error messages (at least to my implementation attempt) to just get the same behavior as with “contextual conversion to bool”?

§ 1.4 R4 revision

Corrected document ID within document. Changes wrt WG14 Adoption (see WG14 N2941 and N3047).

§ 1.5 R3 revision

- provide corrected `assert.h` from usage experience as system header on MacOS Mojave. Compiled many homebrew bundles successfully with it (this is how I found my bugs).
- provide test case code showing C and C++ use with arguments that contain commas.
- resurrect minimal C++ wording change, to show what needs to be done, if WG14 adopts proposed changes to `assert.h`

§ 1.6 R2 revision

- add this history
- provide a new C document number
- added a to-be-discussed in section 5

§ 1.7 R1 revision

- added implementation experiment with local system header
- provide C++-only wording to decouple C from C++ progress

— provide discussion/replies to discussion in working groups in section 5

§ 1.8 R0 initial revision

was discussed by WG21 SG22 (C/C++ liason), WG21 LEWG and WG14

§ 2 Introduction

The `assert()` macro, being a macro, is not very beginner friendly in C++, because the preprocessor only uses parenthesis for pairing and none of the other structuring syntax of C++, such as template angle brackets, or curly braces. This makes it user unfriendly in a C++ context, requiring an extra pair of parentheses, if the expression used, incorporates a comma.

Shafik Yaghmour presented the following C++ code in one of his Twitter quizzes [tweet](#) demonstrating the weakness.

```
#include <cassert>
#include <type_traits>

using Int=int;

void f() {
    assert(std::is_same<int,Int>::value); // a surprisig compile error
}
```

One of the twitter [responses](#) (by [@Static_assert](#)) to the tweet mentioned above, even provided a definition of the `assert` macro that actually is a primitive implementation of what I propose in this paper:

```
#define assert(...) ((__VA_ARGS__)?(void)0:std::abort())
```

In C one needs to be a bit more sophisticated to trigger such a compile error, but nevertheless the C syntax allows for such expression that include commas that are not protected from the preprocessor by parentheses as given by Shafik's [godbolt example](#)

```
#include <assert.h>

void f() {
    assert((int[2]){1,2}[0]); // compile error
    struct A {int x,y};
    assert((struct A){1,2}.x); // compile error
}
```

The current C standard does not even sanction such a compile error to my knowledge, when `NDEBUG` is not defined, since it specifies the `assert` macro to be able to take an expression of *scalar type* which the above non-compiling examples with a comma, I think, are (int in both cases). The C++ standard and working paper refer to C's definition of the `assert` macro in that respect.

§ 3 Remedy

This deficit in the single argument macro `assert()` seems to be very easy to mitigate by providing a `__VA_ARGS__` version of the macro using ellipsis (...) as the parameter.

There exist the option to specify the `assert` macro with an extra name parameter and then the ellipsis. However, I do think this not only complicates its implementation it also complicates its wording, as well as this feature allowing a single argument macro call is not available for C++ versions pre C++20. If the `assert` macro is called without any arguments this will lead to a compile error as it does today. The only difference might be the issued compiler diagnostic

A DIY version can be defined that provides the additional parenthesis needed for the `assert()` macro of today:

```
#define Assert(...) assert((__VA_ARGS__))
```

However, that would be required to be defined and used throughout a project and such is less user friendly than have the standard facility provide such flexibility.

Note: the following *feature* was removed, due to discussions of the R0/N2621 version of this paper. And a mechanism is enforced to detect misuse of the comma operator.

~~In addition the variable argument macro version of `assert` would allow additional diagnostics text, by using the comma operator, such as in~~

```
assert((void)"this cannot be true", -0.0);
```

~~which would otherwise also be required to use an extra pair of parentheses.~~

However, such additional diagnostic strings are better spelled using the `&&` conjugation (thanks to Martin Hořeňovský)

```
assert(idx < vec.size() && "idx is out of range");
```

The proposed solution prevents the use of the comma operator on top-level, to avoid accidentally creating always true assertions like the following:

```
assert(x > 0 , "x was not greater than zero");
```

Those assertions can result from converting a `static_assert` to a regular `assert`.

§ 4 Impact on existing code

On my Mac I changed the system's `assert.h` header to provide variadic macro versions of the `assert(...)` macro for C++ and C. I implemented a mechanism to prevent unintentional use of the comma operator within the macro's arguments. I compiled various software (including LLVM) on my Mac using that changed system header and did not encounter problems, beyond my own bugs that I made in that change. The latter is, why I know that the adapted header was actually used.

When reading the C specification of the semantics of `assert()` one could argue that the macro parameter should already have been variadic, because even in C one can form a scalar expression with a comma that doesn't require balanced parentheses. So WG14 and WG21 might even consider to apply this change as a backward defect fix to previous revisions of the standards. The argument that the existing spec of C11 was broken with that respect was accepted by WG14 and thus the definition of `assert(...)` accepted for C23.

§ 5 Potential liabilities of the proposed change

While sharing a preview of this document and during review of the initial revision in the various online study and work groups addressed, I got several people commenting on it. While some were in favor, there were raised some potential issues that I'd like to share paraphrased below.

There were liabilities that I do not list, because they are already addressed by the initial revision of this paper

- breaking compatibility with C: This is not planned, see R0 version of this paper, however, this revision (R1/2) provide a C++-only wording to enable decoupling progress in WG14 and WG21
- wording only addresses the NDEBUG case (for C++): I provide now more context, the wording changes required for C++ definitely only relevant with the NDEBUG case. However, if desired, we could split adoption of C and C++ in parallel, but providing full `assert` specification in C++, which is done now (in R1/2).

§ 5.1 NDEBUG will allow `assert()` without any arguments

As specified at the moment, the NDEBUG version of `assert(...)` will swallow any macro arguments, even if there is none. I believe this is a small price to pay, since any invalid code that matches a macro argument is allowed today anyway if NDEBUG is set. A more sophisticated specification could use the more-than-one argument variadic macro syntax mentioned below.

§ 5.2 “Contracts will make the 50 year old assert macro obsolete and to not suffer from the macro parsing issue.”

While I appreciate the notion of contracts, I and others think it is worthwhile to make `assert()` more beginner friendly, since professional code bases will have their own versions of precondition checking stuff anyway. While beginners can be shown a universally available feature that is identical to C and could even be ported to older revisions of the standards without breaking existing code.

§ 5.3 “Using the comma operator can be misapplied to an always true assert, if its arguments are formed as for `static_assert(condition, reason)`. This will make wrong code compile that today doesn’t.”

This problem is prevented by my implementation, unless `NDEBUG` is defined, either by defining an identity function taking a single argument (in C) or by using `bool(__VA_ARGS__)`. I do not think we need to update the specification with that respect.

§ 5.4 “Teachability is not improved, because we can teach use extra parentheses today.”

I have a lot of experience in teaching C++ and i believe that using those extra parenthesis is teachable when interacting with students having such a problem, but the surprise and the time it takes to remember this remedy when it hits, is worth the effort to make it more user friendly. Especially, since `assert()` tends to be used as a unit testing framework substitute and thus in C++ the use of templates or initializer lists happens frequently, at least in tests I write.

During discussions in LEWG, WG21/SG22 and WG14 the following issues were raised:

§ 5.5 Will changing `static_assert(condition, "reason")` to `assert(condition, "reason")` compile and silently make the assert never fire. (see 2 above)

ad 2/4. My implementation prevents the use of the comma operator, so the point 2 is clearly taken. Any decent optimizer should also eliminate any call to the inline identity function that is only there to prevent inadvertent use of the comma operator or missing to provide an argument. However, my implementation provides no protection against using zero arguments or the comma operator if `NDEBUG` is defined due to the nature of `assert()` being a macro. The empty argument case could be addressed there as well, but I did not attempt that (yet), because it could lead to side effects or make the solution not backward compatible with older version of the standards, where `assert(arg, ...)` would require at least 2 arguments, whereas in C++20 it only requires a single argument.

§ 5.6 Do you have implementation experience?

As stated above and can be seen in the appendix, I used an adapted `assert.h` on my system and compiled code with it over several months. I found bugs in my implementation, and I cannot guarantee, that there are no corner cases, I missed. But all bugs that I had during that time, stemmed not from the macro being variadic, but my feeble attempts to detect the cases where a comma operator might sneak in and my brain having forgotten C.

§ 5.7 Why don’t you use `bool(__VA_ARGS__)` to prevent comma operator usage?

I did opt for the usage of an identity function, because that approach works both for C++ and C, whereas the suggested remedy `bool(__VA_ARGS__)` would only work for C++. This way, I could prevent implementation divergence between C and C++ of the actual macro replacement. \$ However, with different compilation paths for C and C++ the C++ version will most probably use the construct `bool(__VA_ARGS__)`

§ 5.8 Why don’t you use `assert(arg, __VA_ARGS__)` to prevent zero arguments for the `NDEBUG` case?

As stated above, the feature is only usable in C++20 for at least one argument macros and I want my system header compile with all versions of C++ and C.

I do not have the resources to do a wider spread analysis of the change, and would appreciate help, if such is required before further consideration.

§ 5.9 With the suggested static cast to bool, an enum class type would be viable as truth values, this contrasts their behavior in other boolean contexts

Yes, a static cast to bool might be to simple.

I tried to make it still a compile error by

```
enable_if_t<not is_scoped_enum_v<decltype(__VA_ARGS__)>, bool>(__VA_ARGS__)
```

instead. However, that will create a heavy dependency to type traits and deliver non-user-friendly compile error messages, just to suppress a reasonable but not exactly the same as contextual conversion to bool semantics. I also tried using a function template with static_assert that delivers better error messages, but still gets the dependency.

Fortunately, **Daveed Vandervoorde** and **Jonathan Wakely** provided me with a solution to the problem of checking for contextually convertible to bool types while keeping the simplicity and having no dependencies:

```
#define      assert(...)      ((__VA_ARGS__)      ?      (void)sizeof(bool(__VA_ARGS__))      :  
      (void)__assert_fail(__VA_ARGS__, __FILE__, __LINE__))
```

With that trick, the check for single expression argument is provided by bool(__VA_ARGS__) and the check for contextual conversion to bool is delivered by the first operand to the ternary operator.

In addition it prevents compilability of assert() in C++ even when NDEBUG is not set.

§ 5.10 To be discussed by LWG:

Should the C++-only specification still require that <cassert> and <assert.h> have the same content? This might make sense, but I did not want to put it in yet, before any decision has been made to proceed.

Should we wait for a C23 adaptation paper to make these changes? (when will that happen by whom?)

In my opinion, we can proceed with this without waiting for C23 to be ratified and imported into C++. The proposed changes will have to be made anyway and getting this feature, when we fail to adopt C23, seems to be important enough.

§ 6 Wording for C++ (to be voted on)

The change is relative to n4892. I provide the C++ only change version here. For a change that relies on the WG14 draft standard to change as well, see below. Since C adopted the paper for C23 and C++ didn't for C++23, I expect the later wording to be relevant for C++26 (considering it is to be based on C23).

In [assertions.general] apply the following change (taken from C wording):

- 1 The header <cassert> provides a macro for documenting C++ program assertions and a mechanism for disabling the assertion checks [through defining the macro NDEBUG](#).

In [cassert.syn] change the macro definition as follows:

```
#define assert( E ... ) see below
```

The contents are the same as the C standard library header <assert.h>, except that a macro named static_assert is not defined.

See also: ISO C 7.2

In *[assertions.assert]* perform the following changes.

§ 6.0.1 19.3.2 The assert macro [assertions.assert]

- 1 If NDEBUG is defined as a macro name at the point in the source file where <cassert> is included, the assert macro is defined as

```
#define assert(...) ((void)0)
```

- 2 Otherwise, the assert macro puts a diagnostic test into programs; it expands to an expression of type void which has the following effects:
 - __VA_ARGS__ is evaluated and contextually converted to bool.
 - If the evaluation yields true there are no further effects.
 - Otherwise, the assert macro's expression creates a diagnostic on the standard error stream in an implementation-defined format and calls abort(). The diagnostic contains #__VA_ARGS__ and information on the name of the source file, the source line number, and the name of the enclosing function (such as provided by source_location::current()).
- 3 If __VA_ARGS__ does not expand to an *assignment-expression*, the program is ill-formed.
- 4 The macro assert is redefined according to the current state of NDEBUG each time that <cassert> is included.
- 5 An expression assert(E) is a constant subexpression (16.3.6), if
 - NDEBUG is defined at the point where assert is last defined or redefined, or
 - E contextually converted to bool (7.3) is a constant subexpression that evaluates to the value true.

§ 7 ~~obsolete~~ ~~Wording for C++ if C is changed~~

If C++ will be based on C23 for C++26, the following wording can be applied, relating to the assert(...) specification change for C.

In *[cassert.syn]* change the macro definition as follows:

```
#define assert( E ... ) see below
```

The contents are the same as the C standard library header <assert.h>, except that a macro named static_assert is not defined.

See also: ISO C 7.2

In *[assertions.assert]* no change is be required

§ 7.0.1 19.3.2 The assert macro [assertions.assert]

- 1 An expression assert(E) is a constant subexpression (16.3.6), if
 - NDEBUG is defined at the point where assert is last defined or redefined, or
 - E contextually converted to bool (7.3) is a constant subexpression that evaluates to the value true.

§ 8 Wording for C

This was adopted for C23 and was even considered a bug fix!

These changes are relative to N2573. Those changes were applied for C23 (N3047) and might be considered a specification bug fix, even for previous revisions of the C standard due to the use of “scalar expression” as the

argument to assert's specification if NDEBUG is not defined. Thus, the minimal change for C++ standard proposed above can be used.

In section 7.2 (Diagnostics <assert.h>) change the definition of the assert() macro to use ellipsis instead of a single macro parameter:

- 1 The header <assert.h> defines the assert and static_assert macros and refers to another macro,

NDEBUG

which is not defined by <assert.h>. If NDEBUG is defined as a macro name at the point in the source file where <assert.h> is included, the assert macro is defined simply as

```
- #define assert(ignore) ((void)0)
+ #define assert(...) ((void)0)
```

The assert macro is redefined according to the current state of NDEBUG each time that <assert.h> is included.

- 2 The assert macro shall be implemented as a macro with an ellipsis parameter, not as an actual function. If the macro definition is suppressed in order to access an actual function, the behavior is undefined.

In section 7.2.1 (Program Diagnostics) no change is needed. It is included here for easier reference by reviewers.

§ 8.0.1 7.2.1 Program diagnostics

§ 8.0.1.1 7.2.1.1 The assert macro

Synopsis

- 1

```
#include <assert.h>
void assert(scalar expression);
```

Description

- 2 The assert macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if expression (which shall have a scalar type) is false (that is, compares equal to 0), the assert macro writes information about the particular call that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function – the latter are respectively the values of the preprocessing macros __FILE__ and __LINE__ and of the identifier __func__) on the standard error stream in an implementation-defined format.¹ It then calls the abort function.

Returns

- 3 The assert macro returns no value.

Forward references: the abort function (7.22.4.1).

§ 9 Acknowledgements

Many thanks to Shafik Yaghmour and other Twitterers for inspiring this “janitorial” clean up paper.

Thanks to the reviewers and discussion participants in LEWG, SG22 and WG14.

§ 10 Example implementation (BSD license on MacOS)

The implementation and some simple tests for checking for the non-compilability of calling assert() with the wrong number of arguments are in https://github.com/PeterSommerlad/SC22WG21_Papers/tree/master/workspace/p2264_test_for_assert_dotdotdot_on_my_machine.

Here are the key changes. I introduced


```

#ifdef NDEBUG
#define assert(...) ((void)0)
#else

#ifndef CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#define CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#ifdef __cplusplus
// use bool(__VA_ARGS__)
#else
static inline int __check_single_argument_passed_to_assert(int b) { return b; }
#endif
#endif /* CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED */

#ifdef __cplusplus
#define assert(...) \
    ((void) (( __VA_ARGS__ ) ? ((void)sizeof(bool(__VA_ARGS__))) : __assert (#__VA_ARGS__, \
    __FILE__, __LINE__)))
#else
#define assert(...) \
    ((void) (__check_single_argument_passed_to_assert(__VA_ARGS__) ? ((void)0) : __assert \
    (#__VA_ARGS__, __FILE__, __LINE__)))
#endif

```

Full file:

```

// THIS FILE IS MODIFIED for P2264 (WG21) N2621 & successors (WG14)
// original file modified and tested on MacOS, but OK, because BSD license
// Peter Sommerlad.

/*_
 * Copyright (c) 1992, 1993
 * The Regents of the University of California. All rights reserved.
 * (c) UNIX System Laboratories, Inc.
 * All or some portions of this file are derived from material licensed
 * to the University of California by American Telephone and Telegraph
 * Co. or Unix System Laboratories, Inc. and are reproduced herein with
 * the permission of UNIX System Laboratories, Inc.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 * 3. All advertising materials mentioning features or use of this software
 * must display the following acknowledgement:
 * This product includes software developed by the University of
 * California, Berkeley and its contributors.
 * 4. Neither the name of the University nor the names of its contributors
 * may be used to endorse or promote products derived from this software
 * without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 * @(#)assert.h 8.2 (Berkeley) 1/21/94
 * $FreeBSD: src/include/assert.h,v 1.4 2002/03/23 17:24:53 imp Exp $

```

```

*/

#include <sys/cdefs.h>
#ifdef __cplusplus
#include <stdlib.h>
#endif /* __cplusplus */

/*
 * Unlike other ANSI header files, <assert.h> may usefully be included
 * multiple times, with and without NDEBUG defined.
 */

#undef assert
#undef __assert

#ifdef NDEBUG
#define assert(...) ((void)0)
#else

#ifdef CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#define CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#ifdef __cplusplus
// use bool(__VA_ARGS__)
#else
static inline int __check_single_argument_passed_to_assert(int b) { return b; }
#endif
#endif /* CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED */

#ifdef __GNUC__

__BEGIN_DECLS
#ifdef __cplusplus
void abort(void) __dead2;
#endif /* !__cplusplus */
int printf(const char * __restrict, ...);
__END_DECLS

#ifdef __cplusplus
#define assert(...) \
    ((void) (( __VA_ARGS__ ) ? ((void)sizeof(bool(__VA_ARGS__))) : __assert (#_VA_ARGS__, \
    __FILE__, __LINE__)))
#else
#define assert(...) \
    ((void) (__check_single_argument_passed_to_assert(__VA_ARGS__) ? ((void)0) : __assert \
    (#_VA_ARGS__, __FILE__, __LINE__)))
#endif

#define __assert(e, file, line) \
    ((void)printf ("%s:%u: failed assertion `%s'\n", file, line, e), abort())

#else /* __GNUC__ */

__BEGIN_DECLS
void __assert_rtn(const char *, const char *, int, const char *) __dead2 __disable_tail_calls;
#if defined(__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__) && \
    ((__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__ - 0) < 1070)
void __eprintf(const char *, const char *, unsigned, const char *) __dead2;
#endif
__END_DECLS

#if defined(__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__) && \
    ((__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__ - 0) < 1070)
#define __assert(e, file, line) \
    __eprintf ("%s:%u: failed assertion `%s'\n", file, line, e)
#else
/* 8462256: modified __assert_rtn() replaces deprecated __eprintf() */
#define __assert(e, file, line) \
    __assert_rtn ((const char *)-1L, file, line, e)
#endif
#endif

```

```

#if __DARWIN_UNIX03
#ifdef __cplusplus
#define assert(...) \
    (__builtin_expect(!(__VA_ARGS__), 0) ? __assert_rtn(__func__, __FILE__, __LINE__, #__VA_ARGS__) : (void)sizeof(bool(__VA_ARGS__)))
#else
#define assert(...) \
    (__builtin_expect(!__check_single_argument_passed_to_assert(__VA_ARGS__), 0) ? __assert_rtn(__func__, __FILE__, __LINE__, #__VA_ARGS__) : (void)0)
#endif
#else /* !__DARWIN_UNIX03 */
#ifdef __cplusplus
#define assert(...) \
    (__builtin_expect(!(__VA_ARGS__), 0) ? __assert(__VA_ARGS__, __FILE__, __LINE__) : (void)sizeof(bool(__VA_ARGS__)))
#else
#define assert(...) \
    (__builtin_expect(!__check_single_argument_passed_to_assert(__VA_ARGS__), 0) ? __assert(__VA_ARGS__, __FILE__, __LINE__) : (void)0)
#endif
#endif /* __DARWIN_UNIX03 */

#endif /* __GNUC__ */
#endif /* NDEBUG */

#ifndef _ASSERT_H_
#define _ASSERT_H_

#ifndef __cplusplus
#if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201112L
#define static_assert Static_assert
#endif /* __STDC_VERSION__ */
#endif /* !__cplusplus */

#endif /* _ASSERT_H_ */

```

§ 10.1 C test cases

```

#include "system_assert_h_BSD.h"
#include "system_assert_h_BSD.h"
// NDEBUG not set

struct intpair{ int i,j;};

void checkThatMultipleArgsDontCompile(){
    //assert(1,2,3);
    assert((1,2,3));
}

void checkThatBracesConstructorWithCommasCompiles(){
    assert((int[2]){1,2}[0] == 1);
    //assert((int[2]){1,2}[0] , 1);
    assert((struct intpair){1,2}.j == 1); // false
}

#define NDEBUG
#include "system_assert_h_BSD.h"
// check double inclusion is possible without problems
#include "system_assert_h_BSD.h"
void checkThatMultipleArgsDontCompileNDEBUG(){
    assert(1,2,3); // will compile with NDEBUG set
    assert((1,2,3));
}

void checkThatBracesConstructorWithCommasCompilesNDEBUG(){
    assert((int[2]){1,2}[0] == 1);
    assert((int[2]){1,2}[0] , 1);
    assert((struct intpair){1,2}.j == 1); // false
}

```

```

void runCasserts(){
    checkThatMultipleArgsDontCompileNDEBUG();
    checkThatBracesConstructorWithCommasCompilesNDEBUG();
    checkThatMultipleArgsDontCompile();
    //checkThatBracesConstructorWithCommasCompiles();
}

```

§ 10.2 C++ test cases

```

#include <vector>
#include <memory>

#include "cassert_gcc"
#include "cassert_gcc"
// NDEBUG not set

void checkThatNoArgumentDoesntCompile(){

    //assert();
    // error: too few arguments to function 'constexpr bool
    __check_single_argument_passed_to_assert(bool)
}

void checkThatMultipleArgsDontCompile(){
    //assert(1,2,3); should not compile
    assert((1,2,3));
}

void checkThatBracesConstructorWithCommasCompiles(){
    assert(std::vector<int>{1,2,3,4}.size()==4u);
    //assert(std::vector{1,2,3,4}.size(),4u); // should not compile, comma operator
}

void checkThatContextualConversionToBoolWorks(){
    using vpi = std::vector<std::unique_ptr<int>>;
    using upvpi = std::unique_ptr<std::vector<std::unique_ptr<int>>>;
    upvpi pi = std::make_unique<vpi>();
    assert(pi);
    pi = nullptr;
    assert(!pi);
}

#define NDEBUG
#include "cassert_gcc"
#include "cassert_gcc"

void checkThatNoArgumentDoesntCompileNDEBUG(){
    assert(); // will compile with NDEBUG set
}

void checkThatMultipleArgsDontCompileNDEBUG(){
    assert(1,2,3); // will compile with NDEBUG set
    assert((1,2,3));
}

void checkThatBracesConstructorWithCommasCompilesNDEBUG(){
    assert(std::vector{1,2,3,4}.size()==3u); // false
    assert(std::vector{1,2,3,4}.size(),4u); // will compile with NDEBUG set
}

void runCppasserts(){
    checkThatMultipleArgsDontCompileNDEBUG();
    checkThatBracesConstructorWithCommasCompilesNDEBUG();
    checkThatMultipleArgsDontCompile();
}

```

```
    checkThatBracesConstructorWithCommasCompiles();  
    checkThatContextualConversionToBoolWorks();  
}  
  
extern "C" void runCasserts();  
  
#include <iostream>  
  
int main() {  
    runCasserts();  
    std::cout << "!!!Hello World!!!" << std::endl;  
    runCppasserts();  
}
```

-
1. The message might be of the form: Assertion failed: `_expression_ function _abc_`, file `_xyz_` line `_nnn_`. ↩